RecoMind

Large-Scale Personalized Recommendation Engine

Two-Stage Retrieval & Ranking System · Internship Project – AIML-03

PySpark · TensorFlow Recommenders · FAISS · FastAPI · Redis · React

[Your Name] | AIML-03 Internship Project

The Problem & Motivation

The Challenge of Scale

Scoring millions of catalog items with a deep neural network on every user request is too slow for real-time apps.

Data Leakage Risk

Naive random train/test splits let models peek into future user behavior, inflating offline accuracy.

Cold-Start & Latency

New users lack interaction history, while repeat requests waste compute without caching.

GOAL · End-to-end recommender on MovieLens 25M with <50 ms latency & realistic time-based evaluation

System Design — End-to-End Architecture

OFFLINE PIPELINE

- **PySpark Feature Eng.**
  MovieLens 25M → leak-free features
- **Two-Tower Training**
  User & Item embedding towers
- **Neural Ranking Model**
  Deep ranker on interaction features
- **FAISS Index Build**
  Item embeddings for ANN search
- **Batch Export**
  Models + index → serving layer

ONLINE SERVING LAYER

- **React UI**
  Interactive recommendation cards
- **FastAPI Gateway**
  REST endpoints for recommendations
- **Redis Cache Check**
  Instant hit for repeat users
- **FAISS + Neural Rank**
  Real-time retrieval & ranking
- **Cold-Start Fallback**
  Popularity list for unseen users

System Design — Two-Stage Modeling

STAGE 1 · CANDIDATE RETRIEVAL (SPEED)

Two-Tower Model (TensorFlow Recommenders)

Learns separate embeddings for users and movies in a shared vector space. At inference only the User Tower runs online.
FAISS Index → Approximate nearest-neighbor search narrows thousands of movies to Top-100 candidates in milliseconds.

STAGE 2 · NEURAL RANKING (PRECISION)

Deep Ranking Model

Re-scores the Top-100 candidates using rich features — user activity, item popularity, and genre affinity — to produce the final Top-10 recommendations with higher relevance.

Engineering Challenges & Solutions

01

Memory Bottlenecks on 25M Rows

Added a --sample flag for fast local iteration and offloaded heavy feature aggregations to Apache Spark with Parquet storage.

02

Preventing Data Leakage

Enforced strict chronological train/val/test splits so historical features never leak future ratings.

03

Unseen Users & Serving Latency

Added a Popularity Fallback for cold-start users and Redis caching for instant repeat lookups.

Live Demo / UI Walkthrough

Run the RecoMind React app (or walk through screenshots). Suggested flow:

1 **Known User · user_id: 23**

Triggers Two-Tower + FAISS + Neural Ranker and displays Top-12 personalized movie cards. Note cache-miss latency.

2 **Repeat Request · Cache Hit**

Re-run the same user. Demonstrates Redis Cache Hit ("cached": true) returning in ~4 ms.

3 **New User · user_id: 999999**

Triggers the Cold-Start Fallback ("cold_start": true) and returns a popularity-based list.

Evaluation & Latency Results

0.265

Recall@100

Two-Tower + FAISS

0.182

Recall@50

Two-Tower + FAISS

0.214

NDCG@10

Neural Ranker

0.158

Precision@10

Neural Ranker

SYSTEM LATENCY

~4 ms

Redis Cache Hit

~48 ms

Full Two-Stage Miss

~7 ms

Cold-Start Fallback

0.064

Pop. Baseline R@50

Two-Tower + FAISS achieves nearly 3× higher Recall@50 than the popularity baseline (0.182 vs 0.064).

Next Steps & Future Roadmap

Automated Retraining

Schedule weekly PySpark feature pipelines and model updates using Apache Airflow.

Production Model Serving

Migrate inference to NVIDIA Triton Inference Server for GPU-accelerated dynamic batching.

Real-Time Session Signals

Incorporate in-session clickstreams so recommendations adapt immediately as a user browses.

Content-Based Cold Start

Enhance unseen-user fallbacks using movie genre and metadata embeddings.